Description | Accepted | Editorial | Solutions | Submissions

All Submissions

Accepted 20 / 20 testcases passed

yousab21 submitted at Mar 01, 2026 19:40

Editorial | Solution

Runtime

0 ms | Beats 100.00%

Analyze Complexity

Memory

9.64 MB | Beats 8.96%

Code | C++

```
1 //the way i made 2 queues represnt a stack is
2 // at any time there will be an active queue and an empty queue
3 //the active queue front will always match the stack top (ill explain h
4 //so when poping or returning top the active queue works excacly as a s
5
6 #include<queue>
7 class MyStack {
8 public:
```

View more

More challenges

- 232. Implement Queue using Stacks

Write your notes here

Code

C++ | Auto

```
2  //the way i made 2 queues represnt a stack is
1  // at any time there will be an active queue and an empty queue
3  //the active queue front will always match the stack top (ill explain how i do that in the push method)
1  //so when poping or returning top the active queue works excacly as a stack
2
3  #include<queue>
4  class MyStack {
5  public:
6      queue<int> q1;
7      queue<int> q2;
8      int active;
9      MyStack() {
10         active = 1;
11     }
12
13     void push(int x) {
14         //when you want to push an element to the stack
15         //we push it into the non-active queue so that the element that would be
16         //the top of the stackc is now the front of the non-active queue
17         //then we copy the other elements in the active queue
18         //to tge other queue and make that the active one
19         //and we repeat every time we pusha an element
20         if(active == 1){
21             q2.push(x);
22             while(!q1.empty()){
23                 q2.push(q1.front());
24                 q1.pop();
25             }
26             active = 2;
27         }
28         else if(active == 2){
29             q1.push(x);
30             while(!q2.empty()){
31                 q1.push(q2.front());
32                 q2.pop();
33             }
34             active = 1;
35         }
36     }
37
38     int pop() {
39         int stackTop;
40         if(active == 1){
41             stackTop = q1.front();
```

Saved

Ln 3, Col 40

Testcase | Test Result

Description | Accepted | Editorial | Solutions | Submissions

All Submissions

Accepted 20 / 20 testcases passed

yousab21 submitted at Mar 01, 2026 19:40

Editorial | Solution

Runtime

0 ms | Beats **100.00%**

Analyze Complexity

Memory

9.64 MB | Beats **8.96%**

Code | C++

```cpp
//the way i made 2 queues represnt a stack is
// at any time there will be an active queue and an empty queue
//the active queue front will always match the stack top (ill explain h
//so when poping or returning top the active queue works excacly as a s

#include<queue>
class MyStack {
public:
```

View more

More challenges

- **232. Implement Queue using Stacks**

Write your notes here

Code

C++ | Auto

```cpp
            q1.push(q2.front());
            q2.pop();
        }
        active = 1;
    }
}

int pop() {
    int stackTop;
    if(active == 1){
        stackTop = q1.front();
        q1.pop();
    }
    else if(active == 2){
        stackTop = q2.front();
        q2.pop();
    }
    return stackTop;
}

int top() {
    int stackTop;
    if(active == 1){
        stackTop = q1.front();
    }
    else if(active == 2){
        stackTop = q2.front();
    }
    return stackTop;
}

bool empty() {
return q1.empty() && q2.empty();
}
};

/**
 * Your MyStack object will be instantiated and called as such:
 * MyStack* obj = new MyStack();
 * obj->push(x);
 * int param_2 = obj->pop();
 * int param_3 = obj->top();
 * bool param_4 = obj->empty();
 */
```

Saved

Ln 3, Col 40

Testcase | Test Result

Queue

Description | Accepted | Editorial | Solutions | Submissions

All Submissions

Accepted 22 / 22 testcases passed

yousab21 submitted at Mar 01, 2026 20:14

Editorial | Solution

Runtime

0 ms | Beats **100.00%**

Analyze Complexity

Memory

9.81 MB | Beats **30.51%**

Code | C++

```cpp
//this prblem is very simmiler to the last one except we dont switch active
//we just use a helper stack (i will also explain why in the push function)
#include<stack>
class MyQueue {
    stack<int> active;
    stack<int> helper;
public:
    MyQueue() {
```

View more

More challenges

- 426. Convert Binary Search Tree to Sorted Doubly Linked List
- 1653. Minimum Deletions to Make String Balanced
- 2327. Number of People Aware of a Secret

Write your notes here

Code

C++ | Auto

```cpp
//this prblem is very simmiler to the last one except we dont switch active stacks
//we just use a helper stack (i will also explain why in the push function)
#include<stack>
class MyQueue {
    stack<int> active;
    stack<int> helper;
public:
    MyQueue() {
    }

    void push(int x) {
        //to add an element to the queue and acsess it through a stack we
        //need to add elements such that queue front is always = the stack top
        //so we add elements by following this algorithem :
        //1- reverse all elements into helper stacks so that now the queue back is at the helper top
        //2- add element to the helper top (so it is added after the queue back)
        //3- reverse elements back into active stack so that now the element we added is in the bottom of the stack
        //and the queue front is at the top
        while(!active.empty()){
            helper.push(active.top());
            active.pop();
        }
        helper.push(x);
        while(!helper.empty()){
            active.push(helper.top());
            helper.pop();
        }
    }

    int pop() {
        int queueFront = active.top();
        active.pop();
        return queueFront;
    }

    int peek() {
        int queueFront = active.top();
        return queueFront;
    }

    bool empty() {
        return active.empty();
    }
};
```

Saved

Ln 18, Col 45

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1

Input

Description | Accepted | Editorial | Solutions | Submissions

All Submissions

Accepted 208 / 208 testcases passed

yousab21 submitted at Mar 01, 2026 21:03

Editorial | Solution

Runtime

0 ms | Beats 100.00%

Analyze Complexity

Memory

19.75 MB | Beats 8.71%

Code | C++

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
```

View more

More challenges

- 88. Merge Sorted Array
- 244. Shortest Word Distance II
- 1634. Add Two Polynomials Represented as Linked Lists

Write your notes here

Select related tags 0/5

Code

C++ | Auto

```
class Solution {
private:
    ListNode* resultHead = nullptr;
    ListNode* resultTail = nullptr;

    void addToList(int x){
        if(resultHead == nullptr){
            resultHead = new ListNode(x, nullptr);
            resultTail = resultHead;
        }
        else{
            ListNode* newNode = new ListNode(x, nullptr);
            resultTail->next = newNode;
            resultTail = newNode;
        }
    }

public:
//the way i merge those lists is by following this algorithem:
//1-start both lists from the head
//2-compare the current value in list1 with list2
//3-add the smaller one to the result list and go to the next node in this list
//4-repeat step 2 and 3 until one of the lists are empty
//5-copy the remaining elements from the non empty list
    ListNode* mergeTwoLists(ListNode* list1, ListNode* list2) {
        ListNode* listPtr1 = list1;
        ListNode* listPtr2 = list2;
        while(listPtr1 != nullptr && listPtr2 != nullptr){
            if(listPtr1->val <= listPtr2->val){
                addToList(listPtr1->val);
                listPtr1 = listPtr1 -> next;
            }
            else if(listPtr2->val <= listPtr1->val){
                addToList(listPtr2->val);
                listPtr2 = listPtr2 -> next;
            }
        }
        while(listPtr1 != nullptr){
            addToList(listPtr1->val);
            listPtr1 = listPtr1 -> next;
        }
        while(listPtr2 != nullptr){
            addToList(listPtr2->val);
            listPtr2 = listPtr2 -> next;
        }
        return resultHead;
    }
};
```

Saved

Ln 31, Col 30

Testcase | Test Result

Accepted 168 / 168 testcases passed

yousab21 submitted at Mar 02, 2026 16:41

Editorial | Solution

Runtime

0 ms | Beats 100.00%

Analyze Complexity

Memory

16.00 MB | Beats 90.13%

Code | C++

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
```

View more

More challenges

- 82. Remove Duplicates from Sorted List II
- 1836. Remove Duplicates From an Unsorted Linked List

Write your notes here

Select related tags 0/5

Code

C++ | Auto

```cpp
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */
class Solution {
private:

//this is how i solve this:
//1- if the list is empty just return the empty list
//2- if it is not empty i loop to the element next to last
//3- i check if the value in the next element = the value of the current elmenet
//4- if true then it is a duplicate i delete it and check again for more duplicates
//5- if the next element is not a duplicate i just go to the next iteration
public:
    ListNode* deleteDuplicates(ListNode* head) {
        if (head == nullptr){
         return nullptr;
        }

        ListNode* current = head;
        while(current->next != nullptr){
            if (current->next->val == current->val ){
                ListNode* temp = current->next;
                current ->next = temp->next;
                delete temp;
                temp = nullptr;
            }

            else{
            current = current->next;
            }
         }
        return head;
    }
};
```

Saved

Ln 19, Col 51

Testcase | Test Result

Accepted Runtime: 0 ms

Accepted 166 / 166 testcases passed

yousab21 submitted at Mar 02, 2026 17:49

Editorial | Solution

Runtime

3 ms | Beats 7.96%

Analyze Complexity

Memory

15.90 MB | Beats 14.17%

Code | C++

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
```

View more

More challenges

- 1836. Remove Duplicates From an Unsorted Linked List

Write your notes here

Select related tags 0/5

Code | C++ | Auto

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */

//this one is a bit more confusing ill explain my solution as i go in the code
class Solution {

public:
    ListNode* deleteDuplicates(ListNode* head) {
        //if the list is empty or have one node then it cant be a duplicate just return it
        if (head == nullptr || head->next == nullptr){
            return head;
        }

        //this piece of code tracks if the next node is a duplicate
        //if it is not then just go the next node and continue the loop
        //and we track the node we were on as the "last none duplicate node"
        //also there is one edge case we need to accont for that is if all nodes are duplicates
        //then we delete all nodes so ptr becomes nullptr
        //and then if the loop was likw this " while(ptr->next !=nullptr)"
        //it gives a runtime error because ptr->next is now nullptr->next
        ListNode *ptr = head;
        ListNode *prev = nullptr;
        while( ptr!= nullptr && ptr->next !=nullptr){
            if( ptr->next->val != ptr->val){
                prev = ptr;
                ptr = ptr->next;
                continue;
            }

            //if it is a duplicate then we have to track 3 things before we start deleting elements
            //1- the node just before the duplicate chain (we already did that in the last piece of code)
            //2- the first node in the deuplicate chain i called it "dupStart"
            //3- the node just after the duplicate chain (we get it in the next piece of code)
            else{
                ListNode* dupStart = ptr;

                //this code moves "ptr" to the first node after the loop chain
                //so now we have our "node after duplicates" refrence
                //so we make the node before the dup chain point to the one after it
                //and we also handle the edge case that if the head was a part of the dup chain
                //andwe have to accont for the same edge case as the first loop here too
                while( ptr->next != nullptr && ptr->val == ptr->next->val){
```

Saved Ln 75, Col 37

Testcase | Test Result

Description | Accepted | Editorial | Solutions | Submissions

All Submissions

Accepted 166 / 166 testcases passed

yousab21 submitted at Mar 02, 2026 17:49

Editorial | Solution

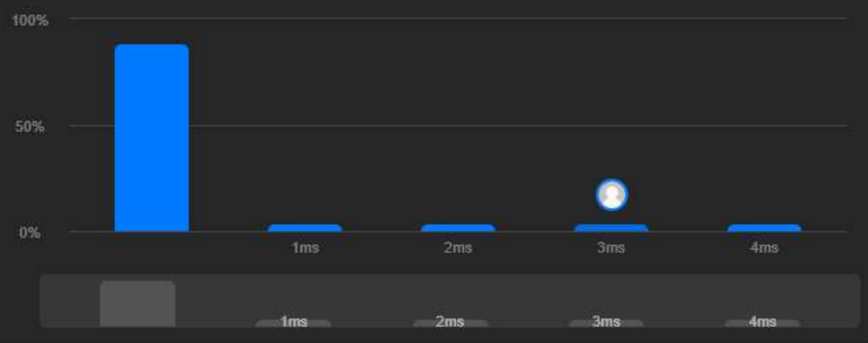
Runtime

3 ms | Beats 7.96%

Analyze Complexity

Memory

15.90 MB | Beats 14.17%



Code | C++

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
```

View more

More challenges

- 1836. Remove Duplicates From an Unsorted Linked List

Write your notes here

Select related tags 0/5

Code

C++ | Auto

```
        //it gives a runtime error because ptr->next is now nullptr->next
        ListNode *ptr = head;
        ListNode *prev = nullptr;
        while( ptr!= nullptr && ptr->next !=nullptr){
            if( ptr->next->val != ptr->val){
                prev = ptr;
                ptr = ptr->next;
                continue;
            }

            //if it is a duplicate then we have to track 3 things before we start deleting elements
            //1- the node just before the duplicate chain (we already did that in the last piece of code)
            //2- the first node in the deuplicate chain i called it "dupStart"
            //3- the node just after the duplicate chain (we get it in the next piece of code)
            else{
                ListNode* dupStart = ptr;

                //this code moves "ptr" to the first node after the loop chain
                //so now we have our "node after duplicates" refrence
                //so we make the node before the dup chain point to the one after it
                //and we also handle the edge case that if the head was a part of the dup chain
                //andwe have to accont for the same edge case as the first loop here too
                while( ptr->next != nullptr && ptr->val == ptr->next->val){
                    ptr = ptr->next;
                }
                ptr = ptr->next;
                if(head == dupStart){
                    head = ptr;
                }
                else{
                    prev->next = ptr;
                    }

                //this code just deletes all nodes from the first node in the dup chain to the
                //fist node after the dup chain
                ListNode *temp = dupStart;
                while (dupStart != ptr){
                    temp = dupStart;
                    dupStart = dupStart->next;
                    delete temp;
                }
            }
        }
        return head;
    }
    //leet code shows that this problem can be solved in 0ms runtime
    //i cant find that solution ill wait for what the instructor has
    //to say about it in the session
};
```

Saved

Ln 75, Col 37

Testcase | Test Result

Description | Accepted | Editorial | Solutions | Submissions

All Submissions

Accepted 1569 / 1569 testcases passed

yousab21 submitted at Mar 02, 2026 18:58

Editorial | Solution

Runtime

1 ms | Beats 45.72%

Analyze Complexity

Memory

76.97 MB | Beats 92.68%

Code | C++

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
```

View more

More challenges

- 43. Multiply Strings
- 445. Add Two Numbers II
- 1634. Add Two Polynomials Represented as Linked Lists

Write your notes here

Select related tags 0/5

Code

C++ Auto

```
//this problem is very similer to another one we had at week1
//same idea with diffrance implementation to represent a linked list
//here is my solution:

class Solution {
    ListNode* resultHead =nullptr;
    ListNode* resultTail= nullptr;

    //this is just a helper function to add elements to the anser list
    void addToResult(int val){
        ListNode* newNode = new ListNode(val, nullptr);
        if(resultHead == nullptr){
            resultHead = newNode;
            resultTail = newNode;
        }
        else{
            resultTail->next = newNode;
            resultTail = newNode;
        }
    }


public:
    ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {
        //first i declare a digit and a carry values to use in the addition operation
        int digit = 0, carry = 0;

        //i calculate the value of the next digit by getting the sum of the matching values in lists + tha carry if it exsists
        //i then add said value in the result array dirictly if it dont exceed 9
        //it does it gets split into a digit that gets added and a carry to be summed up with the next digit
        //i repeat this until one of the lists runs out of elements
        while(l1 != nullptr && l2 != nullptr){
            digit = l1->val + l2->val + carry;
            carry = 0;
            if(digit > 9){
                carry = digit / 10;
                digit = digit % 10;
            }
            addToResult(digit);
            l1 = l1->next;
            l2 = l2->next;
        }

        //if one list is empty and the other stil has elements (one had more digits than the other)
        //i copy the remaining elements to the result while still being mindfull of the carry
        while(l1 != nullptr) {
            digit = l1->val + carry;
            carry = 0;
            if(digit > 9){
                carry = digit / 10;
```

Saved

Ln 80, Col 56

Testcase | Test Result

Accepted 1569 / 1569 testcases passed

yousab21 submitted at Mar 02, 2026 18:58

Editorial | Solution

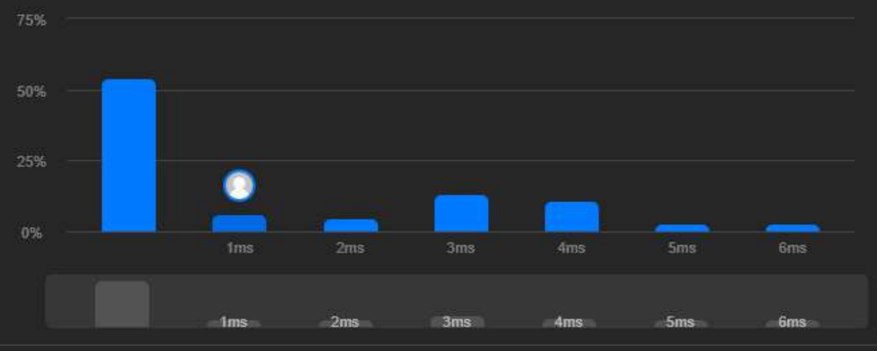
Runtime

1 ms | Beats 45.72%

Analyze Complexity

Memory

76.97 MB | Beats 92.68%



Code | C++

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
```

View more

More challenges

- 43. Multiply Strings
- 445. Add Two Numbers II
- 1634. Add Two Polynomials Represented as Linked Lists

Write your notes here

Select related tags 0/5

Code

C++ Auto

```
    //it does it gets split into a digit that gets added and a carry to be summed up with the next digit
    //i repeat this until one of the lists runs out of elements
    while(l1 != nullptr && l2 != nullptr){
        digit = l1->val + l2->val + carry;
        carry = 0;
        if(digit > 9){
            carry = digit / 10;
            digit = digit % 10;
        }
        addToResult(digit);
        l1 = l1->next;
        l2 = l2->next;
    }

    //if one list is empty and the other stil has elements (one had more digits than the other)
    //i copy the remaining elements to the result while still being mindfull of the carry
    while(l1 != nullptr) {
        digit = l1->val + carry;
        carry = 0;
        if(digit > 9){
            carry = digit / 10;
            digit = digit % 10;
        }
        addToResult(digit);
        l1 = l1->next;
    }
    while(l2 != nullptr) {
        digit = l2->val + carry;
        carry = 0;
        if(digit > 9){
            carry = digit / 10;
            digit = digit % 10;
        }
        addToResult(digit);
        l2 = l2->next;
    }

    //this if condition handels an edge case that the last digit to be pushed was itself greater than 9
    //in that case following the code above we should have already added up the digit
    //so we add the remaining carry into a new node
    if(carry > 0){
        addToResult(carry);
    }
    return resultHead;
  }
};
```

Saved

Ln 80, Col 56

Testcase | Test Result

Description | Accepted | Editorial | Solutions | Submissions

All Submissions

Accepted 30 / 30 testcases passed

yousab21 submitted at Mar 03, 2026 19:02

Editorial | Solution

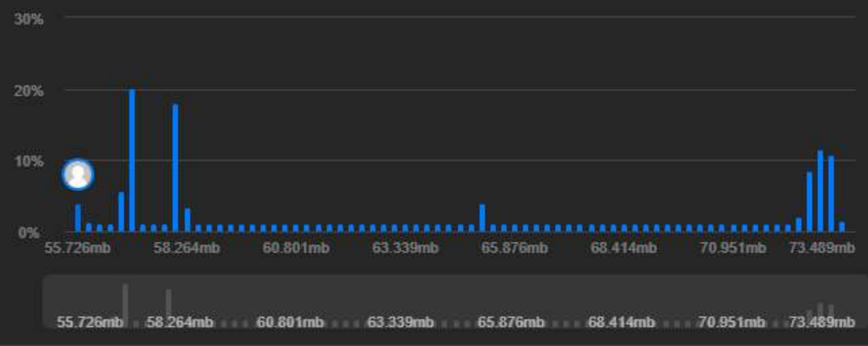
Runtime

2421 ms | Beats 5.15%

Memory

55.94 MB | Beats 96.03%

Analyze Complexity



Code | C++

```cpp
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
```

View more

More challenges

- 75. Sort Colors
- 147. Insertion Sort List
- 2046. Sort Linked List Already Sorted Using Absolute Values

Write your notes here

Select related tags 0/5

Code

C++ | Auto

```cpp
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */


//i tried this problem with bubble sort but leetcode didnt ecept it because of the runtime
//so i searched for faster search algorithems i found one called merge sort that should be a true O(n.log(n))
//but i was not able to implement it so i also found an algorithm called insersion sort that is also O(n^2)
//but it should be alot faster than bubble sort for linked lists
//here is my implementation for it

class Solution {

public:
    ListNode* sortList(ListNode* head) {
        //if we have 0 or 1 node just erturn it as it is
        if(head == nullptr || head->next == nullptr){

            return head;
        }

        //since i will be swapping nodes the head node might get swapped and not be the head any more
        //so this is a reffrence yo keep track of the start of the list
        //also having this makes adding elements to the front of the list a lot eaisier
        ListNode* preHead = new ListNode(0,head);

        //this algorithem works by walking a variable commonly called "key" over every element of the list and
        //commpares every element before it to it
        //find the correct place for the current key
        //this creats an area at the start that with be sorted within itself
        //the elements in this area might not be in thier final position but they are sorted within themself
        ListNode* keyPrev = head;
        ListNode* key = head->next;

        while(key != nullptr) {
            //if the element before the key is smaller than it then we are is said sorted area
            //so we just walk the key until we get to the first unsorted element
            if(key->val >= keyPrev->val){
                keyPrev = key;
                key = key->next;
            }
            else{
```

Saved

Ln 52, Col 115

Testcase | Test Result

Accepted 30 / 30 testcases passed

yousab21 submitted at Mar 03, 2026 19:02

Editorial | Solution

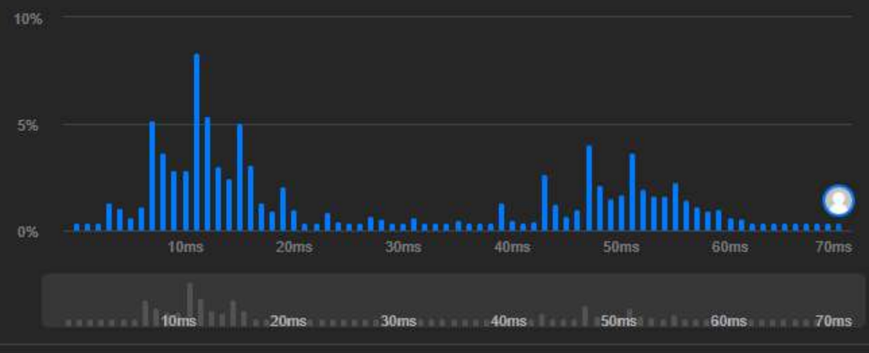
Runtime

2421 ms | Beats 5.15%

Analyze Complexity

Memory

55.94 MB | Beats 96.03%



Code | C++

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
```

View more

More challenges

- 75. Sort Colors
- 147. Insertion Sort List
- 2046. Sort Linked List Already Sorted Using Absolute Values

Write your notes here

Select related tags 0/5

Code

C++ | Auto

```
    //this algorithem works by walking a variable commonly called "key" over every element of the list and
    //commpares every element before it to it
    //find the correct place for the current key
    //this creats an area at the start that with be sorted within itself
    //the elements in this area might not be in thier final position but they are sorted within themself
    ListNode* keyPrev = head;
    ListNode* key = head->next;

    while(key != nullptr) {
        //if the element before the key is smaller than it then we are is said sorted area
        //so we just walk the key until we get to the first unsorted element
        if(key->val >= keyPrev->val){
            keyPrev = key;
            key = key->next;
        }
        else{

        //now we need to find where the current key should fit in the sorted area we have
        //so we walk a scanner on it until if finds a value greter of equal than key (we should add the key right before this value)
        ListNode *scanner = preHead;
        while(scanner->next != key && scanner->next->val < key->val){
            scanner = scanner->next;
        }
        //scanner now is poining to the element the key should be after

        //as this area is sorted and we always track the element before the key
        //we can simply insert the key into its ordered position and link the rest of the list
        //using the variabe used to track the element before the key
        keyPrev ->next = key->next;
        key->next = scanner->next;
        scanner -> next = key;

        //now that the key node is in then correct posisition we can continue walking the key from we left off
         key = keyPrev->next;


        }
    }
    //as the first element can by at the wrong place we cant just return the head of the list
    //as we update the prehead->next pointer when we add an element after it
    ListNode *newHead = preHead->next;
    delete preHead; //now we just delete the prehead beacuse we no longer need it
    return newHead;
  }

};
```

Saved

Ln 52, Col 115

Testcase | Test Result

Linked List

Description | Accepted | Editorial | Solutions | Submissions

All Submissions

Accepted 12 / 12 testcases passed

yousab21 submitted at Mar 03, 2026 19:59

Editorial | Solution

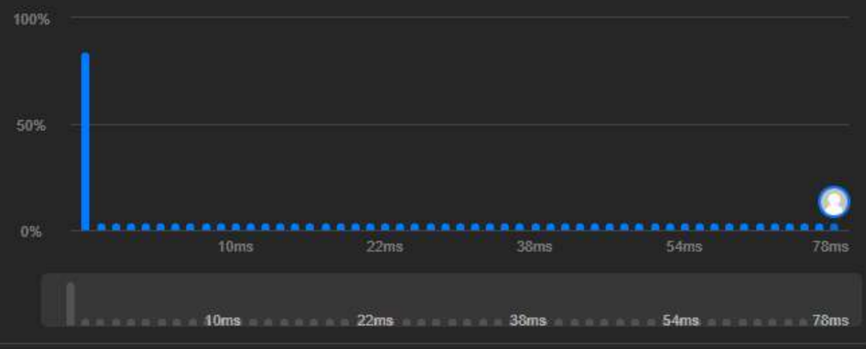
Runtime

1388 ms | Beats **5.02%**

Analyze Complexity

Memory

22.84 MB | Beats **54.54%**



Code | C++

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
```

View more

More challenges

- 2095. Delete the Middle Node of a Linked List
- 2516. Take K of Each Character From Left and Right

Write your notes here

Select related tags 0/5

Code

Submit Ctrl Enter

C++ | Auto

```
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */


class Solution {
public:
    //the resone i want to get the element before the tail not the tail it self
    //is because the the problem wants the tail to be moved so we need to call the function again
    //just to make the newTail->next = nullptr so that i made it return the element before the head
    //to store it in a variable to try to save a fuction call
    ListNode* getBeforeTail(ListNode* head){
        ListNode* temp = head;
        while(temp->next->next != nullptr){
            temp = temp->next;
        }
        return temp;
    }
    void reorderList(ListNode* head) {
        if(head == nullptr || head->next ==nullptr){
            return;
        }

        ListNode *current =head;
        ListNode* next = current->next;

        // here i loop over the elements and add the current tail in between
        //current and next and update the new current to be equal to next
        //which moves it 2 spots in the new list landing at the next element
        //i want to repaat the logic after
        while(current->next != nullptr && current->next->next != nullptr){
            next = current->next;
            ListNode* beforeTail = getBeforeTail(head);
            ListNode* tail = beforeTail->next;
            current->next = tail;
            tail->next = next;
            current = next;
            beforeTail->next = nullptr;
        }
    }
};
```

Saved

Ln 37, Col 43

Testcase | Test Result

Accepted Runtime: 0 ms

Description | Accepted | Editorial | Solutions | Submissions

All Submissions

Accepted 28 / 28 testcases passed

yousab21 submitted at Mar 03, 2026 20:30

Editorial | Solution

Runtime

0 ms | Beats 100.00%

Analyze Complexity

Memory

13.40 MB | Beats 72.27%

Code | C++

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
```

View more

More challenges

- 156. Binary Tree Upside Down
- 234. Palindrome Linked List
- 2807. Insert Greatest Common Divisors in Linked List

Write your notes here

Select related tags 0/5

Code

C++ | Auto

```cpp
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */

//this is just the standerd algorithem in the material
class Solution {
public:
    ListNode* reverseList(ListNode* head) {
        if(head ==nullptr ||head->next == nullptr){
            return head;
        }
        //assing the 3 pointer and walk them next to one another
        //reversing pointer i the porcces until the neode before last
        ListNode* prev = nullptr;
        ListNode* current = head;
        ListNode* next = head->next;

        while(next != nullptr){
            current->next = prev;
            prev = current;
            current = next;
            next=next->next;
        }
        //reverse the last pointer and return the head
        current->next = prev;
        head = current;
        return head;
    }
};
```

Saved Ln 31, Col 55

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

head =

Description | Accepted | Editorial | Solutions | Submissions

All Submissions

Accepted 44 / 44 testcases passed

yousab21 submitted at Mar 04, 2026 19:38

Editorial | Solution

Runtime

0 ms | Beats 100.00%

Analyze Complexity

Memory

11.34 MB | Beats 38.21%

Code | C++

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
```

View more

More challenges

- 109. Convert Sorted List to Binary Search Tree
- 1019. Next Greater Node In Linked List
- 2816. Double a Number Represented as a Linked List

Write your notes here

Select related tags 0/5

Code

C++ | Auto

```
class Solution {

    //i made this function to be able to get a pointer relative to an
    //index in the linked list this will make this alot simpler
    //node than the index starts from 1 in this problem
    ListNode* getPointerToPosition(ListNode* head, int i){
        ListNode* current = head;
        while(i >1 && current != nullptr){
            current = current->next;
            i--;
        }
        return current;
    }

public:
    ListNode* reverseBetween(ListNode* head, int left, int right) {
        //usual edge case if the list is empty or has 1 node
        if(head == nullptr || head->next ==nullptr){
            return head;
        }
        //here i make 3 pointer to point to
        //beforeRev : the last node befoer the sublist i want to reverse
        //startRev : the first node in the sublist i want to revrse
        //afterrev : the fisrt node after the sublist i want to reverse
        //i will use these pointer to glue back the list after reversing tghe sublist
        ListNode* beforeRev = getPointerToPosition(head, left - 1);
        ListNode* startRev = getPointerToPosition(head, left);
        ListNode* afterRev = getPointerToPosition(head, right + 1);
        //this is an edge case for if the beging of the sublist was the first elemtnt
        //in that case we need a preHead pointer to act as a refrence as in this case
        //the head node will be moved
        if(left == 1) {
            beforeRev = new ListNode(0,head);
            startRev = head;
        }

        //these are the normal 3 pointers used to reversed the list identical to the last problem
        //except the current node starts as the start of the sublist i want to reverse
        ListNode* prev = nullptr;
        ListNode* current = startRev;
        ListNode* next = current->next;

        //i start revesing the sublist starting from the start of the sublist
        //until the end of it or until the list ends (edge case if right was the end of the list)
        while(current != afterRev && current!=nullptr){
            next= current->next;
            current->next = prev;
            prev = current;
            current = next;
        }
```

Saved

Ln 51, Col 40

Testcase | Test Result

Description | Accepted | Editorial | Solutions | Submissions

All Submissions

Accepted 44 / 44 testcases passed

yousab21 submitted at Mar 04, 2026 19:38

Editorial | Solution

Runtime

0 ms | Beats 100.00%

Analyze Complexity

Memory

11.34 MB | Beats 38.21%

Code | C++

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
```

View more

Write your notes here

Select related tags 0/5

Code

C++ | Auto

```
        //here i make 3 pointer to point to
        //beforeRev : the last node befoer the sublist i want to reverse
        //startRev : the first node in the sublist i want to revrse
        //afterrev : the fisrt node after the sublist i want to reverse
        //i will use these pointer to glue back the list after reversing tghe sublist
        ListNode* beforeRev = getPointerToPosition(head, left - 1);
        ListNode* startRev = getPointerToPosition(head, left);
        ListNode* afterRev = getPointerToPosition(head, right + 1);
        //this is an edge case for if the beging of the sublist was the first elemtnt
        //in that case we need a preHead pointer to act as a refrence as in this case
        //the head node will be moved
        if(left == 1) {
            beforeRev = new ListNode(0,head);
            startRev = head;
        }

        //these are the normal 3 pointers used to reversed the list identical to the last problem
        //except the current node starts as the start of the sublist i want to reverse
        ListNode* prev = nullptr;
        ListNode* current = startRev;
        ListNode* next = current->next;

        //i start revesing the sublist starting from the start of the sublist
        //until the end of it or until the list ends (edge case if right was the end of the list)
        while(current != afterRev && current!=nullptr){
            next= current->next;
            current->next = prev;
            prev = current;
            current = next;
        }

        //here is glue back the reversed sublist to the rest of the list
        //node that after revesring "prev" becomes the head of the new revered sublist
        // and "startRev" becomes the tail of it
        beforeRev->next = prev;
        startRev->next =afterRev;

        //this condition is for the the same edge case where we start reversing from the head
        //in this case the head of the new list is the head of the revered sublist
        if(left == 1){
            head = prev;
        }

        return head;
    }
};
```

Saved

Ln 51, Col 40

Testcase | Test Result

Description | Accepted | Editorial | Solutions | Submissions

All Submissions

Accepted 232 / 232 testcases passed

yousab21 submitted at Mar 04, 2026 20:23

Editorial | Solution

Runtime

0 ms | Beats 100.00%

Analyze Complexity

Memory

16.36 MB | Beats 64.06%

Code | C++

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
```

View more

More challenges

- 189. Rotate Array
- 725. Split Linked List in Parts

Write your notes here

Select related tags 0/5

Code

C++ | Auto

```
class Solution {
    //i reused the same getBeforeTail function i built for the reorder problem
    ListNode* getBeforeTail(ListNode* head){
        ListNode* temp = head;
        while(temp->next->next != nullptr){
            temp = temp->next;
        }
        return temp;
    }

    //this a function that gets the leanth of the linked list
    //this is very important for a step ill explain later
    int getLength(ListNode* head){
        ListNode* current = head;
        int length = 0;
        while (current != nullptr){
            current = current->next;
            length++;
        }
        return length;
    }

public:
    ListNode* rotateRight(ListNode* head, int k) {
        //usual edge case
        if (head==nullptr  || head->next ==nullptr || k == 0){
            return head;
        }

//================================================//
    int n = getLength(head);                       //
    k = k % n;                                     //
//================================================//
//in my opinion this is the main reason the problem is rated mediam not easy
//by the problem constraintns k could be so big that leetcode wont except the anser
//so we solve this by taking advantage of the fact that rotating the list n times
//just ends up the same as the original list so we need to eliminate multipls of n
//from k until it is smaller than n at firtst i was coding this as a loop until i realized
//the "%" operator does excacly that

        //inside this loop is the logic to make the last node the new
        //head i just repeat it k times
        for(int i = 0; i < k ; i++){
            ListNode* beforeTail = getBeforeTail(head);
            ListNode* tail = beforeTail->next;

            //simple logic to make the tail the new head
            //just make the tail->next point to head
            //make the tail te new haed
            //detach the tail from the before tail node
```

Saved

Ln 46, Col 73

Testcase | Test Result

Description | Accepted | Editorial | Solutions | Submissions

All Submissions

Accepted 232 / 232 testcases passed

yousab21 submitted at Mar 04, 2026 20:23

Editorial | Solution

Runtime

0 ms | Beats 100.00%

Analyze Complexity

Memory

16.36 MB | Beats 64.06%

Code | C++

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
```

View more

More challenges

- 189. Rotate Array
- 725. Split Linked List in Parts

Write your notes here

Select related tags 0/5

Code | C++ | Auto | Submit Ctrl Enter

```
    //this is very important for a step ill explain later
    int getLength(ListNode* head){
        ListNode* current = head;
        int length = 0;
        while (current != nullptr){
            current = current->next;
            length++;
        }
        return length;
    }

public:
    ListNode* rotateRight(ListNode* head, int k) {
        //usual edge case
        if (head==nullptr  || head->next ==nullptr || k == 0){
            return head;
        }

//=============================================//
    int n = getLength(head);                    //
    k = k % n;                                  //
//=============================================//
//in my opinion this is the main reason the problem is rated mediam not easy
//by the problem constraintns k could be so big that leetcode wont except the anser
//so we solve this by taking advantage of the fact that rotating the list n times
//just ends up the same as the original list so we need to eliminate multipls of n
//from k until it is smaller than n at firtst i was coding this as a loop until i realized
//the "%" operator does excacly that

        //inside this loop is the logic to make the last node the new
        //head i just repeat it k times
        for(int i = 0; i < k ; i++){
            ListNode* beforeTail = getBeforeTail(head);
            ListNode* tail = beforeTail->next;

            //simple logic to make the tail the new head
            //just make the tail->next point to head
            //make the tail te new haed
            //detach the tail from the before tail node
            tail->next = head;
            head=tail;
            beforeTail->next= nullptr;
        }
        return head;
    }
};
```

Saved

Ln 46, Col 73

Testcase | Test Result

Description | Accepted | Editorial | Solutions | Submissions

All Submissions

Accepted 41 / 41 testcases passed

yousab21 submitted at Mar 04, 2026 20:37

Editorial | Solution

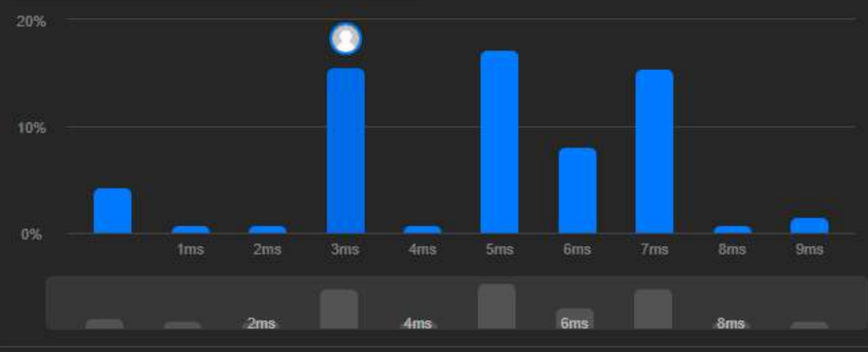
Runtime

3 ms | Beats 95.31%

Analyze Complexity

Memory

12.36 MB | Beats 55.69%



Code | C++

```cpp
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode(int x) : val(x), next(NULL) {}
 * };
 */
```

View more

More challenges

- 203. Remove Linked List Elements
- 2487. Remove Nodes From Linked List
- 3217. Delete Nodes From Linked List Present in Array

Write your notes here

Select related tags 0/5

Code

C++ | Auto

```cpp
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode(int x) : val(x), next(NULL) {}
 * };
 */

//this idea with this problem is we cant delete a specifice node without
//a refrence to its previos node as that will beark the list
//so what we do is we copy the vlaue of the next node and store
//in our refrence node and then linking it to the node after the node we copied
// unlinking the node we copied in the process
class Solution {
public:
    void deleteNode(ListNode* node) {
        node->val = node->next->val;
        node->next = node->next->next;
    }
};
```

Saved

Ln 17, Col 38

Testcase | Test Result